

Resumen Angular — Proyecto Bases

Javier Lanau Gómez · 26 de mayo de 2026

DESCRIPCIÓN GENERAL

Aplicación Angular creada desde cero como ejercicio de aprendizaje de los fundamentos del framework. Consta de dos páginas navegables implementadas como **Standalone Components**, con routing configurado y uso de las nuevas **Signals** de Angular.

ESTRUCTURA DEL PROYECTO

Ruta URL	Componente	Archivo
/	CounterpageComponent	pages/counter/counter.page.components.ts
/hero	HeroPageComponent	pages/hero/hero-page.components.ts

La navegación entre páginas se gestiona desde `app.html` con `<router-outlet>` y enlaces `routerLink`.

PÁGINA 1 — COUNTER

Contador numérico que comienza en **10**. El estado se almacena en una propiedad de clase normal (sin Signals).

- Botones para incrementar: `+1`, `+5`, `+10`
- Botón para resetear el contador a 100
- Usa **event binding** con `(click)="increaseby(n)"`
- Muestra el valor con **interpolación**: `{{ counter }}`

Métodos del componente

Método	Descripción
<code>increaseby(value)</code>	Suma <code>value</code> al contador (por defecto +1)
<code>resetcounter(value)</code>	Fija el contador al valor indicado (por defecto 100)

PÁGINA 2 — HERO

Componente más avanzado que gestiona el estado mediante **Signals**, la nueva primitiva reactiva de Angular.

- Estado inicial: `name = signal('Ironman')`, `age = signal(45)`
- Los valores se leen en la plantilla con `{{ name() }}` (llamada de función)

Métodos del componente

Método	Qué hace	API de Signal
<code>changeHero()</code>	Cambia a Spiderman, 25 años	<code>.set()</code>
<code>changeAge()</code>	Incrementa la edad en +1	<code>.update(age => age + 1)</code>
<code>getHeroDescription()</code>	Devuelve string combinado nombre + edad	Computed manual
<code>changeHeroDescription()</code>	Nombre a mayúsculas y edad +1	<code>.update()</code>
<code>resetForm()</code>	Limpia nombre y edad a vacío/0	<code>.set('')</code>

CONCEPTOS ANGULAR PRACTICADOS

Routing

Configuración de `Routes[]`, `RouterOutlet` y `RouterLink` para navegación sin recarga.

Standalone Components

Componentes con `standalone: true`, sin necesidad de declararlos en un `NgModule`.

Signals

`signal()`, `.set()` y `.update()` para gestión de estado reactivo moderno.

Data Binding

Interpolación `{{ }}` y event binding `(click)` para conectar lógica y plantilla.

Separación en páginas

Estructura `src/app/pages/` con cada página en su propia carpeta (HTML, CSS, TS).

Signals vs propiedades

Counter usa propiedad normal; Hero usa Signals — comparación directa de ambos enfoques.

EVOLUCIÓN DEL PROYECTO (COMMITTS)

Commit	Fecha	Qué se hizo
<code>inicio</code> <code>proyecto</code>	Antes del 26/05	Creación del proyecto Angular base
<code>task 1</code>	26/05 — 15:03	Creación de la página Hero completa (TS + HTML + CSS) y configuración de las rutas
<code>commit</code>	26/05 — 15:32	Refactorización de HeroPageComponent, limpieza y simplificación del código
<code>standalone</code>	26/05 — 15:43	Componentes marcados como <code>standalone: true</code> y corrección de rutas
<code>solucion</code>	26/05 — 15:45	Ajustes finales en la barra de navegación (<code>app.html</code>) y <code>app.ts</code>

